프로토콜 매뉴얼 및 구현 참조서

SWP2P

AVR MCU용 오픈드레인 병렬 피어 투 피어 버스

프레임 필드, 에지 사이클, 상태 머신 및 소스 코드 대응

문서 범위

이 매뉴얼은 이 저장소의 SWP2P 소스 코드에 구현된 프로토콜을 설명합니다. 이전 v1.0 문서와 현재 코드의 내용이 다르면 현재 구현을 기준으로 합니다. 이 문서는 구현 참조 자료이며, 모든 다중 노드 충돌 상황이 하드웨어에서 검증되었다는 뜻은 아닙니다.

문서 버전	v1.1.5 구현 매뉴얼
대상 구현	Arduino Uno / ATmega328P, AVR 레지스터 및 인터럽트
기본 데이터 폭	W4, 프리셋 `PRESET_W4_D4_D7`
기본 버스트 용량	`SWP2P_MAX_BURST = 16`
프레임 형식	DEST(6) + SRC(6) + BURST(1), 이어서 LEN 또는 DATA
저작권	Sierrastdio, 2026

Contents

1 목적과 용어 **2**

2 물리 신호와 에지 규칙 **2**

3 프레임 형식 **2**
3.1 논리 프레임 2
3.2 버스트 길이 인코딩 3

4 W4 파형과 명명된 사이클 **3**
4.1 단일 바이트 경로 4
4.2 버스트 경로 4

5 중재와 노드 필터링 **4**

6 클럭 생성과 동적 속도 **4**

7 구현 대응표 **4**
7.1 상태 레지스터 4
7.2 프레임 변수와 플래그 4

8 애플리케이션 인터페이스와 사용 순서 **5**

9 제한과 검증 참고사항 **5**

10 소스 파일 **6**

1 목적과 용어

SWP2P는 소프트웨어로 구현된 다중 노드 버스입니다. 중앙 패킷 라우팅 마스터 없이 각 노드가 전송을 요청할 수 있습니다. 지정된 한 노드가 공통 클럭을 공급하지만, 클럭 공급 노드가 어느 노드가 전송할지를 결정하지는 않습니다. 노드들은 BUSY가 활성화된 동안 헤더를 전송하며 중재합니다.

이 문서는 *와이어 프로토콜*과 *구현 상태 머신*을 구분합니다. 와이어 필드는 DATA 선에서 관측되는 비트를 말합니다. 상태 번호와 변수명은 현재 AVR 소스 코드를 가리킵니다. 별도 표기가 없으면 각 필드와 데이터 바이트는 최상위 비트(MSB)부터 전송합니다.

호환성 주의

v1.1.5의 압축 헤더는 기존 DEST/SRC 2바이트 헤더와 와이어 호환되지 않습니다. 같은 버스의 모든 노드는 동일한 형식의 구현을 사용해야 합니다. 버전 협상이나 구형 프레임 자동 감지는 지원하지 않습니다.

2 물리 신호와 에지 규칙

Table 1: Uno 구현에서 사용하는 신호

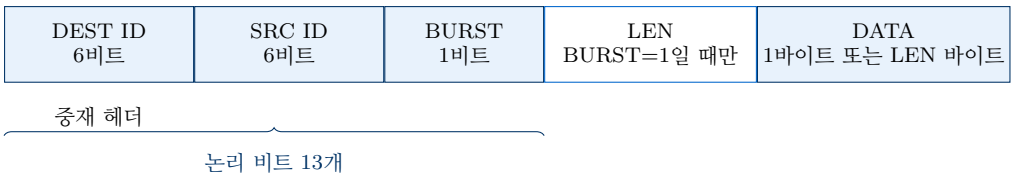
신호	Uno 핀	사용 노드	구현상 의미
CLK 입력	D2 / PD2 / INT0	모든 노드	논리 변화 인터럽트입니다. 핀 레벨을 확인해 상승 에지는 송신/갱신 단계, 하강 에지는 수신/되읽기 단계로 처리합니다.
CLK 출력	D9 / PB1 / OC1A	클럭 공급 노드	Timer1 CTC 하드웨어가 OC1A를 토글합니다. 다른 노드는 이 신호를 D2에 연결합니다.
BUSY	D3 / PD3 / INT1	공유 선	LOW 활성 신호입니다. 송신자는 LOW로 구동해 활성화하고 입력/하이 임피던스로 전환해 해제합니다.
ACK	D8 / PB0	공유 선	LOW 활성 신호입니다. 수신자는 마지막 데이터 바이트 뒤 ACK를 활성화하고 BUSY가 유휴 상태로 돌아오면 ACK를 해제합니다.
DATA	프리셋에 따라 다름	공유 병렬 선	오픈드레인 방식입니다. 노드는 0을 보낼 때 LOW로 구동하고 1을 보낼 때 핀을 해제합니다.

DATA, BUSY, ACK는 방향 레지스터를 바꿔 LOW로 구동하거나 선을 해제하며, 외부 풀업 저항이 HIGH를 만듭니다. 클럭 출력은 DATA 오픈드레인 함수가 아니라 Uno의 Timer1 OC1A 출력에서 생성됩니다. 소스 코드는 D2와 D3의 인터럽트를 설정하지만 버스 풀업은 설정하지 않습니다.

각 와이어 청크에서 송신자는 CLK 상승 에지에 DATA를 갱신합니다. 수신자는 대응하는 하강 에지에서 DATA를 샘플링합니다. 송신자도 중재 중에는 하강 에지에서 DATA를 되읽습니다. 따라서 CLK 한 주기마다 프리셋 폭의 청크 하나를 전송하며, 주기의 절반은 갱신, 나머지 절반은 샘플링에 사용됩니다.

3 프레임 형식

3.1 논리 프레임



와이어 헤더는 다음 필드의 연결입니다.

$$\boxed{\mathrm{DEST}[5:0]}\ \boxed{\mathrm{SRC}[5:0]}\ \boxed{\mathrm{BURST}}.$$

헤더 끝에는 프리셋 폭의 정수 배 청크를 채우기 위한 0 패딩이 붙습니다. 별도의 시작 바이트는 없습니다. BUSY가 활성화되면 프레임이 시작되고 청크 경계는 설정된 데이터 폭으로 정해집니다.

Table 2: 헤더 필드와 인코딩

필드	폭	의미
DEST	6비트	목적지 노드 ID는 0 62입니다. 값 63(`0x3F`)은 브로드캐스트용입니다.
SRC	6비트	송신 노드 ID는 0 62입니다. 생성자는 ID를 6비트로 마스킹하고 로컬 ID 63은 62로 변경합니다.
BURST	1비트	0이면 DATA 1바이트가 바로 뒤따릅니다. 1이면 LEN 필드 뒤에 LEN만큼의 DATA 바이트가 뒤따릅니다.
헤더 패딩	0 7비트	마지막 프리셋 청크를 채우는 후행 0 패딩입니다. 독립 필드는 아닙니다.
LEN	컴파일타임 폭	BURST=1일 때만 존재하며 `len - 2`를 인코딩합니다. 비트 폭은 `SWP2P_MAX_BURST`에 따라 달라집니다.
DATA	바이트당 8비트	데이터는 MSB부터 전송하며 선택한 프리셋 폭에 따라 청크로 나뉩니다.

브로드캐스트는 목적지 값이며 송신자 ID로 쓰지 않습니다. 모든 수신 노드가 브로드캐스트 패킷을 FIFO에 저장하고 ACK 상태에 도달한 각 노드가 공유 ACK 선을 LOW로 구동합니다. 송신자는 ACK가 활성화된 사실은 알 수 있지만 어느 수신 노드가 응답했는지는 구분할 수 없습니다.

3.2 버스트 길이 인코딩

`sendBurst()`는 `len > 1`일 때만 헤더의 버스트 비트를 설정합니다. 버스트의 LEN 전송값은 `len - 2`입니다. 수신자는 디코딩한 값에 2를 더하므로 LEN=0은 2바이트를 뜻합니다. LEN 비트 수는 설정된 최대 버스트를 표현하는 최소 폭으로 정하고 DATA 청크의 정수 배로 올림합니다.

기본 최대 길이 16바이트에서는 0 14 값을 표현하면 되므로 `LEN_BITS=4`입니다. W4에서는 니블 1개이며 LEN 패딩은 없습니다. 와이어 버스트 길이는 2 16바이트입니다. `sendBurst()`에 길이 1을 전달하면 단일 바이트 경로를 사용하며 BURST=0으로 전송합니다.

Table 3: 기본 프레임 청크 수(`SWP2P_MAX_BURST=16`)

프리셋 폭	와이어 헤더 비트	헤더 클럭 수	LEN 비트/클럭 수	DATA 바이트당 클럭	비고
W1	13	13	4 / 4	8	헤더 패딩 없음.
W2	14	7	4 / 2	4	헤더 끝에 패딩 1비트.
W4	16	4	4 / 1	2	헤더 끝에 패딩 3비트.
W8	16	2	4 / 1	1	헤더 끝에 패딩 3비트.

W4 헤더는 기존 16비트 헤더를 니블 단위로 올림했을 때와 마찬가지로 여전히 4클럭을 차지합니다. 기본 최대 버스트 길이에서는 압축 LEN이 기존 8비트 LEN의 W4 전송(2클럭)보다 버스트당 1클럭을 절약합니다. 이는 와이어 사이클 수 비교이며, 안전한 CLK 상한이 높아진다는 보장은 아닙니다.

4 W4 파형과 명명된 사이클

아래 그림은 논리적인 W4 버스 시퀀스를 보여줍니다. 각 청크는 CLK 한 주기입니다. CLK 상승 에지에서 값을 구동/갱신하고 하강 에지에서 샘플링합니다. 버스트 예시 헤더는 DEST=`0x02`, SRC=`0x01`, BURST=1입니다. 논리 헤더 13비트에 패딩 3비트를 더하면 `0x0818`이며 니블 `0, 8, 1, 8`(비트열 `0000 1000 0001 1000`)로 전송됩니다.

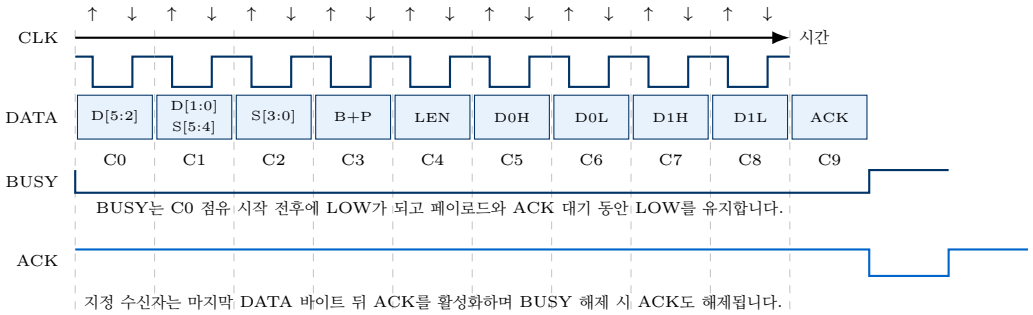


Figure 1: W4 버스트 전송 예시(기본 최대 16바이트). C0 C3은 패딩된 중재 헤더, C4는 LEN이며 데이터 바이트마다 2클럭이 필요합니다. D=DEST, S=SRC, B=BURST, P=후행 패딩 3비트입니다. D0H/D0L은 DATA[0]의 상위/하위 니블입니다. 그림에는 예시 데이터 2바이트만 표시했으며 프레임 길이가 고정된 것은 아닙니다.

네 번째 헤더 니블은 최상위 비트가 BURST이고 뒤의 세 비트는 0 패딩입니다. C1에서 앞의 두 비트가 DEST를 끝내고 나머지 두 비트가 SRC를 시작합니다. 청크 경계와 필드 경계가 일치하지 않을 수 있습니다.

사이클 이름	물리 동작	현재 소스의 상태/변수 동작
S	BUSY가 활성화되고 다음 CLK 상승 에지에서 송신자가 첫 헤더 청크 C0를 구동합니다.	TX: `_txState=1` (PENDING)가 2 (ARB)로 바뀝니다. `_txArbReg`, `_arbMyChunk`, `_arbChunkCount`를 초기화합니다. RX는 INT1의 `_onBusyEdge()`에서 시작합니다.
C0...C(A-1)	헤더/중재 청크입니다. 상승 에지에서 DATA를 갱신하고 하강 에지에서 되읽으며 수신기가 헤더를 누적합니다.	TX는 `_arbMyChunk`와 `_readDataChunk()`를 비교합니다. RX는 `_rxAddrByte`와 `_rxSrcByte`에 헤더를 누적합니다. `A=ADDR_CHUNKS`입니다.
L0...L(K-1)	버스트에서만 LEN 청크를 전송합니다.	TX 상태 3은 `_txDataReg`, `_txDataChunkCount`를 사용합니다. RX 상태 3은 `_rxDataByte`에 누적한 뒤 `_rxLen`을 설정합니다. `K=LEN_CHUNKS`입니다.
D(i,0)...	데이터 바이트 i를 `8/DATA_WIDTH`개 청크로 나눠 전송합니다.	TX 상태 4는 `_txIdx`를 증가시킵니다. RX 상태 4는 `_rxDataByte`를 조립한 뒤 `_fifoPush()`로 데이터와 송신자 ID를 저장합니다.
ACK	수신기는 마지막 바이트 뒤 ACK를 LOW로 활성화하고 송신자는 정해진 수의 CLK 에지 동안 응답을 기다립니다.	RX 상태 5가 PB0/D8을 구동합니다. TX 상태 7은 ACK를 확인하고 `FLAG_ACK_FAILED`를 지우거나 타임아웃으로 설정합니다.
IDLE	송신자가 BUSY를 해제해 선이 HIGH로 돌아갑니다.	INT1 유휴 경로가 busy 플래그를 지우고 ACK 및 RX 파서를 초기화하며 TX는 송신 상태를 해제합니다.

4.1 단일 바이트 경로

단일 바이트 전송은 BURST=0이며 LEN이 없습니다. W4에서는 헤더 니블 4개 뒤에 DATA[7:4], DATA[3:0]이 오고 ACK 처리로 이어집니다. 데이터 길이가 항상 1바이트이므로 종료 지점을 알기 위해 LEN이 필요하지 않습니다.

4.2 버스트 경로

버스트에서는 BURST=1이며 수신기는 먼저 설정된 수의 LEN 청크를 읽습니다. `_rxLen = LEN + 2`를 디코딩한 뒤 해당 바이트 수만큼 수신하고 ACK 단계로 이동합니다. 바이트마다 ACK를 보내지는 않습니다. 완성된 바이트는 도착 즉시 FIFO에 저장되므로 애플리케이션의 `readBytes()` 호출은 프레임 수신이 끝나기 전에 일부 FIFO 데이터만 읽을 수도 있습니다.

5 중재와 노드 필터링

여러 송신자가 경쟁하면 각 노드는 헤더 청크를 보내고 공유 DATA 버스를 되읽습니다. DATA 선을 해제(1 전송)했는데 실제 버스가 LOW인 것을 감지한 송신자는 DATA와 BUSY를 해제하고 TX$_L$$OST\BUSY\Gamma 4\D\$ti.(0x\Gamma(X\Delta|¸ < \D\¸fiøXT|i.

클럭 비공급 노드는 DEST가 일치하지 않는다는 사실을 판별할 만큼 헤더를 읽은 뒤 RX 파서를 멈춥니다. 전기적으로 버스 연결을 끊는 것은 아닙니다. DATA 버스가 오픈드레인이므로 선을 해제한 노드는 다른 노드의 전송을 방해하지 않습니다. 클럭 공급 노드는 목적지가 아닌 프레임의 LEN 및 데이터 청크 수를 계속 추적해 중재용 클럭 속도로 복귀할 시점을 파악합니다.

> **구현상 제한**
>
> 코드는 멀티비트 프리셋 청크를 비트 마스크 불일치 검사로 비교합니다. W1에서는 이 방식이 단순합니다. W2/W4/W8에서 동시에 충돌하면 각 선의 중재 결과가 섞여 단일 송신자만 승리하도록 보장하기 위해 청크 전체의 사전식 비교가 필요한 경우가 생길 수 있습니다. 이 매뉴얼은 현재 구현을 기록한 것이며, 해당 동작을 수정하거나 하드웨어에서 검증하기 전까지 모든 다중 노드 동시 중재가 검증되었다고 간주하면 안 됩니다.

6 클럭 생성과 동적 속도

`begin(true, clkFreq, dataFreq)`로 시작한 노드는 Timer1 CTC와 OC1A(D9)로 CLK를 생성합니다. 나머지 노드는 `begin(false)`를 사용해 D2의 외부 클럭을 따릅니다. 기본 클럭을 생략하면 기준 속도는 100 kHz이며, 프로젝트 예제는 중재 45 kHz와 페이로드 50 kHz를 사용합니다.

소스 코드는 압축 헤더가 끝나고 페이로드 구간이 시작될 때 `_clkGoFast()`를 호출합니다. 버스트에서는 LEN과 DATA가 `dataFreq`로 동작하고 단일 프레임에서는 DATA가 `dataFreq`로 동작합니다. 마지막 페이로드 뒤에는 `_clkGoSlow()`가 `clkFreq`를 ACK/유휴 단계의 속도로 복구합니다. Timer1 설정은 클럭 공급 노드만 변경합니다. 주파수 값만으로 안정적인 상한이 보장되지는 않으며 ISR 실행 시간과 전기적 신호 정착 시간이 실용 속도를 제한합니다.

7 구현 대응표

7.1 상태 레지스터

상태 값은 매크로를 통해 ATmega328P의 범용 입출력 레지스터에 직접 저장됩니다. `_txState`는 `GPIOR1`, `_rxState`는 `GPIOR2`입니다. 상태 전이는 `_onClkEdge()` 및 `_onBusyEdge()` 내에서 처리됩니다.

Table 4: `SWP2P.h`의 TX/RX 상태

TX	이름	의미	RX	이름	의미
0	IDLE	대기 중인 송신 없음	0	IDLE/FILTERED	파서 동작 없음, 목적지 불일치 필터링 포함
1	PENDING	시작/클럭 에지 대기	1	HEADER	압축 DEST/SRC/BURST 헤더 수집
2	ARB	헤더 청크 송신/비교	3	LEN	버스트 길이 수신
3	LEN	버스트 길이 송신	4	DATA	자기 목적지 DATA 수신
4	DATA	페이로드 바이트 송신	5	ACK	수신 완료 후 ACK 활성화
6	LOST	중재 패배, 유휴 시 재시도	6	DISCARD	클럭 공급 노드가 목적지 외 DATA를 프레임 끝까지 계산
7	WAIT_ACK	ACK 또는 타임아웃 대기			

7.2 프레임 변수와 플래그

Table 5: 소스 변수와 프로토콜 의미 대응

변수	저장 형식	역할
`_nodeId`	정적 바이트	생성자에서 0 62로 정규화하는 로컬 송신자 ID입니다.
`_txDestId`	정적 바이트	하위 6비트는 DEST입니다. 7번 비트는 `sendBurst()`가 내부적으로 사용하는 표식이며 와이어 헤더 바이트로 전송되지 않습니다.
`_txLen`, `_txIdx`	정적 바이트	요청된 페이로드 길이와 현재 TX 바이트 인덱스입니다.

변수	저장 형식	역할
`_txBuffer[]`, `_txData`	정적 바이트 배열	복사된 버스트 페이로드 및 단일 바이트 페이로드입니다. `sendBurst()`는 반환 전에 데이터를 복사합니다.
`_txArbReg`	16비트	왼쪽 정렬된 13비트 헤더와 프리셋 후행 패딩입니다. MSB부터 시프트해 전송합니다.
`_arbChunkCount`	바이트	첫 헤더 청크를 구동한 뒤 남은 헤더 청크 수입니다.
`_arbMyChunk`, `_arbChunksSent`	바이트	되읽기 중재에 사용하는 로컬 청크와 지금까지 구동한 청크 수입니다.
`_txDataReg`, `_txDataChunkCount`	바이트	LEN/DATA 시프트 레지스터와 남은 청크 수입니다.
`_rxAddrByte`, `_rxSrcByte`	바이트	두 변수는 16비트 압축 헤더 수신 누산기를 구성합니다. 디코딩 후 `_rxSrcByte`는 송신자 ID 저장에 재사용됩니다.
`_rxDataByte`	바이트	LEN 누산기 또는 현재 DATA 바이트 누산기입니다.
`_rxLen`, `_rxByteIdx`, `_rxChunkCount`	바이트	디코딩한 데이터 바이트 수, 현재 바이트 인덱스, 남은 청크 수입니다.
`_rxFifo[]`, `_rxSrcFifo[]`	FIFO	데이터와 송신자 ID를 같은 인덱스로 저장하는 병렬 큐입니다.
`GPIOR0` 플래그	레지스터	송신 중, busy, 자기 목적지 패킷, ACK 실패 플래그입니다(비트 0, 1, 3, 5).
`_ocrArb`, `_ocrData`	16비트	기준 및 페이로드 속도에 사용하는 Timer1 비교 값입니다.

TX 경로의 헤더 인코딩은 다음과 같습니다.

```
header = (DEST << 7) | (SRC << 1) | BURST;
wireHeader = header << (16 - 13);
```

수신기는 패딩된 와이어 헤더를 모은 다음 `ADDR_PADDING_BITS`만큼 오른쪽으로 시프트한 뒤 DEST, SRC, BURST를 추출합니다.

8 애플리케이션 인터페이스와 사용 순서

Table 6: 주요 공개 API

API	동작
`begin(clkIsOutput, clkFreq, dataFreq)`	핀과 인터럽트를 초기화하고 필요하면 Timer1 클럭 출력/속도를 설정합니다.
`send(dest, byte)`	1바이트 프레임(BURST=0)을 논블로킹으로 큐에 넣습니다.
`sendBurst(dest, ptr, len)`	데이터를 TX 저장 공간에 복사해 프레임을 큐에 넣습니다. len=1이면 단일 바이트 형식을 사용합니다.
`sendBurst(dest, buffer)`	현재 `SWP2PBuffer` 내용을 큐에 넣습니다.
`available()` / `read()`	데이터 바이트와 송신자 ID를 확인하거나 읽습니다.
`readBytes(data, src, max)`	현재 FIFO에서 읽을 수 있는 바이트를 꺼냅니다. 프레임 전체가 완료될 때까지 기다리지는 않습니다.
`isSending()` / `isBusy()`	로컬 송신 진행 여부와 공유 BUSY 상태를 반환합니다.
`isAckFailed()`	가장 최근 ACK 타임아웃 플래그를 반환한 뒤 지웁니다.

1. 애플리케이션 메모리에 전송 데이터를 준비합니다.
2. `send()` 또는 `sendBurst()`를 호출합니다. 반환값 `true`는 로컬 TX 상태 머신에 접수되었음을 뜻하며 전달 완료를 의미하지는 않습니다.
3. 라이브러리가 중재, 프레임 송신, ACK 대기를 처리하는 동안 애플리케이션은 계속 실행될 수 있습니다.
4. `isSending()`이 해제될 때까지 확인한 뒤 `isAckFailed()`를 호출해 결과를 확인합니다.
5. 수신 노드에서는 FIFO에서 데이터와 송신자 ID를 꺼냅니다. FIFO 깊이는 `SWP2P_FIFO_DEPTH`(기본값 16)이며 패킷 큐가 아니라 바이트 큐입니다.

9 제한과 검증 참고사항

- `SWP2P_MAX_BURST`는 1 255로 설정할 수 있습니다. TX 저장 용량과 컴파일타임 LEN 폭을 결정하며, 수신기는 설정값을 넘는 디코딩 길이를 거부합니다.
- 사용 가능한 노드 ID는 0 62이고 63은 브로드캐스트 DEST로 예약됩니다. 버스의 모든 노드는 프리셋, 프레임 버전, 최대 버스트 설정, 호환되는 클럭 동작을 공유해야 합니다.
- 프레임에는 CRC/체크섬이 없습니다. ACK는 수신기가 수신 경로의 끝까지 도달했음을 확인할 뿐 모든 데이터 비트가 정확했음을 보증하지는 않습니다.
- RX FIFO는 바이트 단위의 유한 큐입니다(기본 16개). FIFO가 가득 차면 `_fifoPush()`가 바이트를 조용히 버리므로 애플리케이션에서 제때 읽어야 합니다.
- 그림은 논리적인 에지 순서를 나타냅니다. 실제 상승/하강 시간, 최대 안정 주파수, 동시 중재 동작은 실제 보드, 배선, 풀업, 빌드로 검증해야 합니다.

- Uno에서 D9는 Timer1 OC1A, D8은 PB0/ACK입니다. 클럭 공급 노드에서는 D9를 DATA로 지정하는 프리셋을 함께 쓸 수 없습니다. D2와 D3은 CLK/BUSY 전용 핀입니다. 예제의 D10 입력은 W4의 D4 D7과 별개입니다.

10 소스 파일

- `src/SWP2P.h`: 공개 API, TX/RX 상태 머신, ISR 핸들러, 타이머 속도 전환.
- `src/SWP2Ppreset.h`: 프리셋 핀 매핑, 오픈드레인 DATA 동작, 프레임 폭 특성.
- `src/SWP2P.cpp`: 정적 저장 공간, FIFO 구현, Timer1 설정.
- `src/SWP2Pbuffer.h`: 애플리케이션 측 고정 용량 버퍼 유틸리티.